ChatGPT

Katab 오케스트레이터 분석 및 차세대 아키텍처 제안

현재 프로젝트 구조 분석

본 리포트는 업로드된 `katab-main` 패키지의 내용을 기반으로 작성되었다. 프로젝트는 QA 자동화 도구 **Katab**의 서버/CLI 부분으로, 다음과 같은 구조를 갖는다.

전체 구조

- **패키지 구성**: 최상위에는 `packages/recorder`와 `packages/device-manager`가 있으며, `recorder`가 오케스트레이터와 대시보드 서버 역할을 한다. `device-manager`는 모바일 디바이스 제어를 위한 코드다.
- **데이터 폴더**: `graphs`, `folders`, `groups`, `auth-profiles` 등은 사용자가 만든 시나리오, 폴더 구조, 사용자 인증 프로파일 등을 JSON 혹은 이미지로 저장한다. `scenarios_steps.csv`에는 시나리오별 단계가 정리되어 있다.
- **README/CLI**: 루트의 `README.md`는 Katab을 CLI로 사용하는 방법(녹화, 편집, 실행 등)을 설명한다. CLI 명령(`pnpm katab record…`)을 통해 Web/App 테스트 시나리오를 생성하고 실행할 수 있다.

오케스트레이터 구현

`packages/recorder/src/orchestrator`에는 백엔드 오케스트레이터 모듈이 정의되어 있다. 주요 구성요소는 다음과 같다.

- **OrchestratorService**: SQLite 데이터베이스와 Redis 큐(BullMQ)를 사용하여 실행 요청(Run)과 시나리오 실행(ScenarioRun)을 관리하고, 작업을 큐에 enqueue한다. 작업 생성 시 `createRun` 메서드가 Run 레코드를 만든 뒤 시나리오들을 BullMQ 큐에 enqueue한다.
- **Queues(BullMQ)**: BullMQ는 Redis 위에 구축된 분산 작업 큐로, 생산자(Orchestrator)가 작업을 추가하면 하나 이상의 **Worker**가 이를 처리한다. BullMQ 공식 문서에서는 "작업은 **다른 머신에 있는 여러 Worker**들에 의해 처리될 수 있다" [1] 고 명시하여, 현재 아키텍처가 단일 서버일지라도 BullMQ는 복수의 Worker(예: 여러 PC)에 작업을 배포할 수 있도록 설계되어 있다.
- **DB 스키마**: `src/orchestrator/db/schema.sql` 안에는 runs, scenario_runs, resources, schedules, planned_runs 등 다양한 테이블이 정의되어 있다. 특히 `schedules`와 `planned_runs` 테이블은 크론 표현식, 단일 실행, 다른 스트림 후 실행(AFTER) 같은 스케줄 유형을 지원하며, 실행 상태, 중복 실행 정책, lookahead 등의 필드를 포함한다.
- **API 라우트**: `src/orchestrator/api-routes.ts`는 REST API (`/api/v2/runs`, `/api/v2/resources`, `/api/v2/queue-stats` 등)를 정의해 대시보드 서버와 오케스트레이터가 통신하도록 한다. webhooks 및 webhook 이벤트 저장 기능도 포함한다.
- **Webhook/Discord/Slack 렌더러**: 작업 상태 변화 시 특정 Webhook을 호출하는 구현이 포함돼 있다.

현재의 한계

- **단일 서버**: `recorder` 패키지는 SQLite DB와 로컬 Redis 인스턴스를 사용한다. 모든 실행 요청, 스케줄 생성, Worker가 한 서버에서 수행된다.
- **단일 테넌시**: 계정/테넌트 개념이 코드에 녹아있지 않다. `auth-profiles` 폴더에 JSON 파일들이 존재하지만, 각 계정별로 시나리오를 격리하거나 실행 수를 제한하는 기능이 없다.

- **스케줄 실행**: 오케스트레이터 내부에 크론 기반 스케줄러가 구현돼 있지만, 스케줄이 로컬 서버에만 영향을 준다. 클라우드 환경에서 별도의 Runner에게 작업을 분산하는 기능이 없다.

향후 목표 정리

사용자 요구사항을 정리하면 다음과 같다.

1. **Multi-Server 분리**: 기존 Katab 서버는 하나의 프로세스였는데, 이를 두 가지 역할로 분리하고자 한다.
2. **클라우드 서버(Orchestrator/Dashboard)**: 스케줄 관리, 실행 요청 생성, 계정 관리 및 UI를 제공한다. 여러 계정을 지원하는 멀티테넌트 애플리케이션이다. 여기서 스케줄의 RUN 요청을 생성하면 로컬 PC로 전달해야 한다.
3. **로컬 PC 서버(Runner)**: 사용자의 PC에서 실행되며, 클라우드가 요청한 테스트 시나리오를 실행한다. 이는 Katab CLI의 replay 기능 또는 기존 Worker 로직을 기반으로 한다.
4. **오케스트레이터 기능 분리**: 현 Orchestrator는 Run/Resource/Schedule/Webhook/Signal 기능을 하나의 모듈에서 처리한다. 향후에는 **계정별 서비스**로 분리해 각 테넌트에서 스케줄, 실행, 웹훅, 시그널을 독립적으로 관리할 수 있어야 한다. 이는 다중 테넌트 아키텍처를 의미한다. 다중 테넌트는 하나의 애플리케이션이 여러 고객(테넌트)을 서비스하지만 각자의 데이터는 논리적으로 격리되는 구조를 의미한다 [2].
5. **스케줄 기능 우선 제공**: 초기 목표는 스케줄 관리 기능을 클라우드에서 제공하고, 스케줄 실행 명령만 로컬 PC로 전달하는 것이다. 이후 녹화, 시나리오 편집, 웹훅/시그널 등 대시보드 기능을 클라우드에 올려 사용자 경험을 완성할 계획이다.
6. **계정 및 커맨드 관리**: 클라우드 서버는 각 계정에 대한 **시나리오 목록, 스케줄 목록, 실행 내역**, 그리고 몇 개의 명령(Run) 요청이 로컬 PC로 전달됐는지를 관리해야 한다. 로컬 Runner는 자신에게 도착한 작업을 계정별로 집계해 보고할 수 있어야 한다.
7. **시나리오 확인**: 각 계정이 보유한 시나리오를 확인하고 편집/다운로드할 수 있는 API 및 UI가 필요하다.

제안하는 새 아키텍처

아래 다이어그램은 향후 분리된 구조를 요약한다. 텍스트로 설명하면 다음과 같다.

1. **클라우드 계층 (Katab Orchestrator Cloud)**
2. **Tenant API Gateway**: 각 계정(테넌트)에 대해 인증/인가(JWT, OAuth2)를 수행하고 적절한 서비스로 라우팅한다.
3. **Account Service**: 계정 생성, 사용자 관리, 역할/권한 관리, 과금/사용량 집계를 담당한다.
4. **Scenario Service**: 시나리오 파일을 저장(S3 등), 버전 관리, 계정별 목록 조회/업로드를 처리한다. 변경 사항이 있을 경우 Runner에게 동기화 알림을 전송한다.
5. **Schedule Service**: 기존 `schedules` / `planned_runs` 테이블을 확장해 다중 테넌트를 지원한다. Cron/At/After 스케줄을 생성하고, 실행 시간에 맞춰 **Run Service**에 실행 요청을 발행한다. BullMQ의 `QueueScheduler`를 사용해 미스파이어 정책과 중복 실행 정책을 적용한다.
6. **Run Service (Orchestrator)**: 계정별로 Run을 생성하고 ScenarioRun을 분해하여 큐에 넣는다. 기존 `OrchestratorService`를 리팩터링해 테넌트 정보를 포함하고, BullMQ 큐를 Redis Cluster로 확장한다. BullMQ는 여러 Worker가 다른 머신에서 동일한 큐를 소비할 수 있도록 설계되어 있다 [1].
7. **Webhook/Signal Service**: 각 계정별로 웹훅을 설정하고, Run 상태 변화 시 이벤트를 발행한다. Signal은 사용자가 실행 중 시나리오에 수동으로 입력해야 하는 데이터를 전달하는 채널로 확장할 수 있다.

8. **Dashboard UI**: React/Vue 등의 SPA로 구현하여 녹화/편집/스케줄/실행 현황을 표시한다. REST/GraphQL API를 통해 위 서비스들과 통신한다.

9. **메시지 브로커 & 데이터베이스**

10. **Redis (BullMQ)**: 중앙 큐로 Redis Cluster를 사용한다. BullMQ는 작업을 큐에 추가하고 다른 머신의 Worker가 이를 소비할 수 있도록 해, **클라우드-로컬 간 작업 전달**을 간단하게 구현해 준다 [1].

11. **Persistent DB**: 다중 테넌트를 지원하는 관계형 데이터베이스(PostgreSQL/MySQL)를 사용한다. 각 테이블에 `tenant_id` 컬럼을 추가해 데이터 격리를 보장하거나, 계정마다 별도 데이터베이스를 생성하는 **Single Application, Multiple Database** 모델을 택할 수 있다 [2]. 간단하게는 기존 SQLite 스키마를 PostgreSQL로 변환하고 `tenant_id`를 추가하는 방향을 추천한다.

12. **로컬 PC 서버 (Runner)**

13. **Runner 프로세스**: 사용자의 PC에서 실행되며, 클라우드에 등록된 `resource_id` 및 `tenant_id`를 사용해 인증한다. Redis/HTTPS를 통해 BullMQ 큐를 구독하고 자신에게 할당된 시나리오 실행(Job)을 소비한다.

14. **Scenario Executor**: 기존 Katab CLI의 `replay` 기능을 사용하여 로컬 브라우저나 모바일 디바이스에서 테스트를 실행한다. 실행 결과를 클라우드의 Run Service로 보고하며, HTML/JSON 리포트를 업로드한다.
15. **Local API**: 사용자가 로컬에서 Runner 상태를 확인하고 일시 중단/재시작할 수 있는 간단한 UI 혹은 CLI를 제공한다. 또한 스케줄 동기화나 시나리오 다운로드를 위해 Scenario Service와 통신한다.
16. **Security**: Runner는 계정별 API 토큰을 사용해 자신의 작업만 가져온다. 또한 실행 환경을 샌드박스화하여 악의적인 시나리오 실행을 제한한다.

단계별 구현 계획

1단계: 아키텍처 준비

1. **데이터베이스 리팩터링**: 기존 SQLite 스키마를 PostgreSQL/MySQL로 이전하고 `tenant_id` 컬럼을 추가한다. `schedules`, `runs`, `scenario_runs`, `resources`, `planned_runs`, `webhooks` 등 모든 테이블이 계정 식별자를 포함해야 한다. 테넌트마다 다른 DB를 사용하는 모델도 고려할 수 있으나, 초기에는 하나의 DB 내부에 `tenant_id` 컬럼을 두는 것이 관리에 편하다.
2. **인증/인가 도입**: JWT 기반의 인증 시스템을 도입하여 각 요청에 `tenant_id`와 사용자 권한을 포함한다. Account Service에서 계정 생성 및 사용자 초대/삭제를 지원한다.
3. **메시지 브로커 구성**: Redis Cluster 혹은 AWS ElastiCache 같은 관리형 Redis를 준비한다. BullMQ를 동일하게 사용하되, `queueName:tenantId:platform` 형태로 큐를 분리해 각 계정의 작업을 격리한다.

2단계: 서비스 분리 및 클라우드 기능 개발

1. **Run Service 추출**: `OrchestratorService` 코드를 `run-service`로 분리한다. `createRun` 메서드에서 `tenant_id`를 받아 Run/ScenarioRun을 생성하고, 작업을 각 테넌트용 BullMQ 큐에 enqueue한다. API 라우트를 `/api/v2/tenants/{tenantId}/runs`와 같이 변경한다.
2. **Schedule Service 개발**: 기존 `schedules.ts`와 `planned_runs` 로직을 기반으로 스케줄링 서비스를 만든다. Cron/At/After 스케줄 생성 시 `tenant_id`를 지정하고, 만료 정책/중복 정책을 설정할 수 있도록 한다. 스케줄이 도래하면 Run Service의 API를 호출해 Run을 생성한다.
3. **Scenario Service**: 시나리오 파일 및 폴더 구조를 객체 저장소(S3)나 DB에 저장한다. 각 시나리오 메타데이터에 `tenant_id`, `id`, `name`, `created_at`, `updated_at` 등을 저장하고, 파일은 버전 관리한다. 클라우드 대시보드에서 업로드/다운로드/편집이 가능하도록 API를 제공한다.
4. **Webhook/Signal Service**: 각 테넌트의 webhook 설정을 저장하고, Run 상태가 변경될 때 적절한 Webhook을 호출한다. Signal 서비스는 수행 중 수동 입력이 필요한 경우 클라우드 UI를 통해 사용자가 값을 입력하면 Runner에게 전달하도록 메시지 채널을 제공한다.

5. **Dashboard UI**: 리액트 기반 SPA를 개발하여 로그인, 계정 선택, 시나리오 관리, 스케줄 관리, 실행 모니터링, 보고서 확인 등을 제공한다. 초기에는 스케줄 생성과 실행 모니터링 기능만 제공하고, 이후 녹화/편집 화면을 웹으로 마이그레이션한다.

3단계: 로컬 Runner 구현 및 배포

1. **Runner 클라이언트 개발**: Node.js 기반으로 `@katab/runner` 패키지를 만들고, Redis 연결 및 BullMQ Worker를 초기화한다. `tenant_id`와 `resource_id`를 사용해 자신의 큐(`{tenantId}:{platform}`)를 구독하고, Job 데이터를 수신한다.
2. **Scenario 실행 모듈 통합**: 기존 `replay` 명령을 모듈화하여 Runner에서 호출할 수 있도록 한다. Job payload에는 `scenarioIds`, `options`, `authProfileId` 등이 포함된다. 실행이 끝나면 결과를 클라우드에 업로드한다.
3. **상태 리포팅**: Runner는 실행 중 상태, 큐 상태, 최근 결과를 주기적으로 Run Service에 전송한다. 클라우드는 이를 기반으로 대시보드에서 로컬 PC의 상태를 표시한다.
4. **배포/업데이트 전략**: Runner는 일회성 설치 프로그램 혹은 Docker 이미지로 배포할 수 있으며, 자동 업데이트 기능을 제공한다.

4단계: 계정/사용량 관리와 모니터링

1. **사용량 집계**: 각 `run_id`를 계정과 연계하여 클라우드는 월별 실행 횟수, 사용 시간, 리소스 소비량을 계산한다. 이를 토대로 과금 또는 제한 정책을 구현할 수 있다.
2. **모니터링/로그**: Prometheus/Grafana 또는 APM을 사용해 Run Service, Schedule Service, Runner의 상태를 모니터링한다. BullMQ에는 기본적인 큐 상태 조회 API가 있으므로 이를 활용한다.
3. **보안 감사**: Run 실행 시 시나리오와 입력 데이터가 민감할 수 있으므로 암호화와 감사 로그를 기록한다.

결론

현재 Katab 프로젝트는 단일 서버/싱글 테넌트 구조로 되어 있지만, 오케스트레이션 로직은 BullMQ를 기반으로 하고 있어 **다중 Worker/다중 머신 지원이 가능**하다 [1]. 이를 활용해, 클라우드 서버에서 스케줄 관리와 계정 관리를 담당하고, 각 사용자는 로컬 PC에 Runner를 설치하여 자신의 시나리오를 실행하는 **분산형 멀티테넌트 아키텍처**로 확장할 수 있다. 멀티테넌트 모델에서는 하나의 애플리케이션이 여러 고객을 서비스하면서도 데이터와 구성을 격리하도록 설계된다 [2]. 제안된 단계적 계획을 통해 기존 코드를 최대한 재활용하면서도 새로운 요구사항(계정별 서비스 분리, 스케줄 우선 제공, 명령 전달)을 충족하는 시스템을 구축할 수 있을 것이다.

---

[1] Introduction | BullMQ
https://docs.bullmq.io/guide/introduction

[2] Multi-Tenant Architecture: How It Works, Pros, and Cons | Frontegg
https://frontegg.com/guides/multi-tenant-architecture